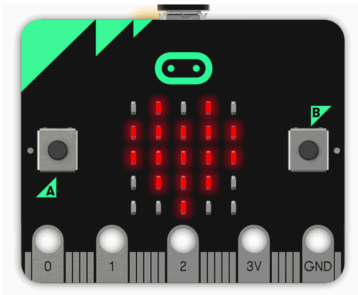
Lab 4: Operating Systems



Hilary Term 2026

Mark van der Wilk — Lecturer & Adaptations
Mike Spivey — Lecturer & Course Designer

Contents

1. Introduction 1
2. Tasks 2

ⓘ **Much to read, not as much to do. Read *before* the scheduled lab session!**

1. Introduction

This lab introduces micro:bian, a very simple embedded operating system kernel. The directory `lab4-microbian` contains the following files:

File Name	Description
`Makefile`	Build script
`hardware.h`	Header file with layout of I/O registers
`lib.c, lib.h`	Library with implementation of printf
`startup.c`	Startup code
`nRF51822.ld`	Linker script
`microbian.c, microbian.h`	Operating system
`mpx-m0.s`	Context switch code for Cortex-M0
Device drivers	
`serial.c`	Serial port
`timer.c`	Timer
`i2c.c`	I2C bus (incl. accelerometer)
`radio.c`	2.4GHz radio
Example programs	
`ex-heart.c`	Heart and primes
`ex-echo.c`	Echo lines from keyboard
`ex-race.c`	Relative process speeds
`ex-today.c`	Mutual exclusion
`ex-level.c`	Accelerometer-based spirit level

`ex-remote.c`	Remote button presses

There are several example programs for you to experiment with:

- `heart` is the ultimate version of the electronic Valentine's card, containing independent, concurrent processes for displaying the beating heart and printing the romantic list of prime numbers on the serial port.
- `echo` is a simple test program for the UART driver. You can type lines of text on the keyboard, with echoing and line editing, and they are printed back when you press Return. - `race` is a demonstration of scheduling uncertainty: one process increments a counter while another periodically prints its value. The precise sequence of values printed depends on when the processes are scheduled.
- `today` is an exercise in mutual exclusion: two politicians repeatedly spout their slogans, but they cannot be understood unless an interviewer intervenes to make them take turns.
- `level` is an electronic spirit level. It uses the I2C bus to talk to the accelerometer chip on the micro:bit, and it displays a single moving pixel that responds when the board is tilted.
- `remote` is a radio-based remote control. If two or more micro:bits in the same room are running the program, then pressing button A or B on one of them will cause all the others to display A or B respectively.

To support these programs, the operating system kernel (in `microbian.c`) is augmented with drivers for the UART (in `serial.c`), a system timer (`timer.c`), the I2C bus that links the processor to the on-board accelerometer and magnetometer (`i2c.c`), and the 2.4GHz packet radio intergrated into the microcontroller (`radio.c`). For simplicity, the header file `microbian.h` declares in one place the routines provided by all these modules.

Typing `make` or choosing Build>Make as usual compiles all the example programs into hex files ready for download to the micro:bit. With one of the example programs open, you can choose Build>Upload me to upload the corresponding hex file to the micro:bit.

2. Tasks

Requirements for S

1. Today: Run the program and observe the output. Then introduce an interviewer process that makes the two politicians speak in turn. One solution has each politician passing its slogans to the interviewer; another has the interviewer giving permission to a politician to speak until they indicate they have finished.
2. Race: Run the program and observe the output. Then try swapping the two calls the `start()` from `init()`. Why does this affect the action of the program? Although difficult to predict precisely in advance, the results printed are actually consistent from run to run: why is that?
3. Heart: Try removing the system call that gives the display process a higher priority than the primes process. Then start searching for primes at 1000000 or 10000000 instead of 2. Observe the results, then reinstate the priority. Why should the display process have a higher priority than the primes process?

Requirements for S+

- Design an interesting multi-person application that uses the radio to communicate. As configured, the radio module can broadcast packets containing up to 32 bytes of payload. For point-to-point communication, you could embed a destination address in each packet, and have each micro:bit ignore messages that were not addressed to it. One idea is to implement the chain reaction game.

OR

- Do something creative.

Extra exercises:

1. Make a driver process for the hardware random number generator. Write a program that shows random dice rolls on the display whenever a button is pressed. Alternatively, there is an onboard sensor that measures the temperature of the processor die, giving an answer in quarters of a degree Celsius. It generates an interrupt when data is ready, but then suspends itself until started again. Write a device driver for it.
2. Construct a test program to measure the time taken to send and receive a message as the length of an output pulse. Experiment to find the combination of circumstances that makes this quickest: does sending the message with `sendrec` help, and why?